Bruktbilprispredikator

Navn: Martin Olsnes Helland
Dato: 30.10.2026

1: BESKRIV PROBLEMET

OMFANG / SCOPE

I dette prosjektet ønsket jeg å lage en nettside som kunne gi et enkelt prisestimat på en bruktbil. Brukeren fyller inn opplysninger om bilen, som merke, modell, årsmodell, kilometerstand og motor, og får deretter en estimert pris.

Jeg valgte bruktbilpriser fordi det er et problem der mange forskjellige egenskaper påvirker resultatet samtidig. Før jeg startet tenkte jeg for eksempel at årsmodell og kilometerstand kom til å være viktige, men datasettet inneholdt også informasjon om merke, modell, motor, girkasse og skadehistorikk. Derfor ønsket jeg å teste om en maskinlæringsmodell kunne finne sammenhenger mellom disse variablene og pris bedre enn enkle faste regler.

En vanlig måte å finne bruktbilpris på er å søke etter tilsvarende biler på nettet og sammenligne annonser. Problemet er at det kan være vanskelig å finne biler som faktisk er sammenlignbare. To biler av samme modell og årsmodell kan ha veldig forskjellig kilometerstand eller motor.

Målet mitt ble derfor å lage en enkel løsning der brukeren kan fylle inn informasjonen én gang og få et raskt estimat. Jeg forventet ikke at modellen skulle finne den nøyaktige markedsprisen, men ønsket å undersøke hvor godt det var mulig å predikere pris ut fra variablene i datasettet.

Prosjektet består av flere steg. Jeg startet med å undersøke og rydde datasettet, og hentet deretter ut nye variabler fra motorbeskrivelsene. Etter dette delte jeg dataene i trenings- og testdata og sammenlignet en baseline med tre forskjellige regresjonsmodeller. Modellen som fikk best resultat ble til slutt koblet til en Streamlit-nettside.

METRIKKER

For å kunne sammenligne modellene brukte jeg tre hovedmetrikker: RMSE, MAE og R^2.

RMSE, eller Root Mean Squared Error, måler hvor stor forskjell det er mellom de faktiske prisene og prisene modellen predikerer. Store feil får ekstra stor betydning i denne metrikken. Det betyr at noen få veldig store feil kan trekke RMSE mye opp. Lavere RMSE betyr bedre resultat.

MAE, eller Mean Absolute Error, er den gjennomsnittlige absolutte forskjellen mellom den faktiske prisen og den predikerte prisen. Denne er lettere å forstå i praksis fordi resultatet bruker samme enhet som bilprisen. Hvis MAE for eksempel er 20 000 dollar, betyr det omtrent at modellen bommer med 20 000 dollar i gjennomsnitt.

R^2 sier noe om hvor mye av variasjonen i prisene modellen klarer å forklare. En verdi nærmere 1 er bedre, mens en lav verdi betyr at mange forhold som modellen ikke kjenner til også påvirker prisen.

Jeg brukte også en enkel baseline-modell. Denne modellen predikerer alltid medianprisen fra treningsdataene. Baseline-modellen gir et enkelt sammenligningsgrunnlag. Hvis maskinlæringsmodellene ikke gjør det bedre enn denne, vil de ikke gi særlig mye ekstra verdi.

2: DATA

Datasettet jeg brukte kommer fra Kaggle og heter Regression of Used Car Prices – Playground Series, Season 4, Episode 9.

Datasettet inneholder totalt 188 533 biler og 13 opprinnelige kolonner. Variabelen modellen skal predikere er price.

Gjennomsnittsprisen i datasettet er omtrent 43 878 dollar, mens medianprisen er 30 825 dollar. Den billigste bilen har en pris på 2 000 dollar, mens den høyeste registrerte prisen er 2 954 083 dollar. Forskjellen mellom gjennomsnitt og median viser at det finnes noen svært dyre biler i datasettet som trekker gjennomsnittet opp.

Datasettet inneholder blant annet informasjon om merke, modell, årsmodell, kilometerstand, drivstofftype, motor, girkasse, skadehistorikk, clean title og pris.

Til modellen brukte jeg følgende numeriske egenskaper:

- Årsmodell
- Kilometerstand
- Hestekrefter
- Motorstørrelse
- Antall sylindre

I tillegg brukte jeg disse kategoriske egenskapene:

- Merke
- Modell
- Drivstofftype
- Girkasse
- Skadehistorikk
- Clean title

Feature engineering

Motorinformasjonen var en av delene av datasettet jeg måtte bearbeide før den kunne brukes på en god måte. engine-kolonnen inneholdt tekststrenger som for eksempel:

300.0HP 3.0L V6 Cylinder Engine Gasoline Fuel

Det betydde at hestekrefter, motorstørrelse og antall sylindre ikke allerede lå som egne numeriske variabler.

Jeg brukte derfor regulære uttrykk i Python til å hente ut disse verdiene. Fra eksempelet over fikk jeg:

- 300 hestekrefter
- 3,0-liter motor
- 6 sylindre

Jeg delte derfor opp motorbeskrivelsen slik at hestekrefter, motorstørrelse og antall sylindre kunne brukes som egne numeriske variabler. Det var også her jeg oppdaget et problem med datasettet: motorbeskrivelsene hadde ikke alltid samme format. Derfor klarte jeg ikke å hente ut alle verdiene, og dette førte til manglende data for blant annet hestekrefter og antall sylindre.

I stedet for å fjerne alle disse radene valgte jeg å fylle inn manglende numeriske verdier med medianen fra treningsdataene.

Manglende verdier

Datasettet inneholder en del manglende verdier.

De opprinnelige kolonnene med flest manglende verdier var:

Kolonne	Manglende verdier	Prosent
Clean title	21 419	11,36 %
Fuel type	5 083	2,70 %
Accident	2 452	1,30 %

Siden motorbeskrivelsene ikke alltid hadde samme format, klarte programmet ikke å hente ut alle verdiene.

Omtrent:

- 17,64 % av verdiene for hestekrefter manglet
- 3,55 % av verdiene for motorstørrelse manglet
- 20,08 % av verdiene for antall sylindre manglet

Jeg valgte å ikke fjerne alle radene med manglende informasjon, siden dette ville ført til at mye data forsvant.

For numeriske verdier brukte jeg medianen til å fylle inn manglende verdier. For kategoriske verdier brukte jeg den vanligste kategorien.

Til Ridge Regression standardiserte jeg også de numeriske variablene og brukte one-hot encoding på de kategoriske variablene.

For Random Forest og Gradient Boosting brukte jeg ordinal encoding.

Jeg brukte Pipeline og ColumnTransformer fra scikit-learn slik at preprocessing og selve modellen holdes samlet. Dette gjør det enklere å passe på at nøyaktig samme preprocessing blir brukt når en bruker senere sender inn nye data gjennom nettsiden

Treningsdata og testdata

Datasettet ble delt inn i:

- 80 % treningsdata
- 20 % testdata

Jeg brukte random_state=42 slik at jeg kunne få samme datasplitt igjen dersom programmet ble kjørt på nytt.

Dette ga omtrent:

- 150 826 trenings-eksempler
- 37 707 test-eksempler

Treningsdataene ble brukt til å lære modellene, mens testdataene ble brukt til å evaluere modellene etter treningen. Dette er viktig fordi det gir et mer realistisk bilde av hvordan modellen fungerer på biler den ikke har sett før.

Personvern og etiske hensyn

Det er relativt få personvernproblemer med den nåværende løsningen fordi brukeren kun skriver inn informasjon om bilen. Nettsiden spør ikke etter navn, adresse, telefonnummer eller annen personlig informasjon.

Det finnes likevel noen utfordringer med selve datasettet. Enkelte merker og modeller finnes sannsynligvis mye oftere enn andre. Det kan føre til at modellen fungerer bedre på vanlige biler enn på sjeldne modeller.

Den faktiske prisen kan avhenge av forhold som ikke finnes i datasettet, for eksempel bilens fysiske tilstand, servicehistorikk, ekstrautstyr og hvordan markedet ser ut.

3: MODELLERING

Jeg sammenlignet tre maskinlæringsmodeller i tillegg til en enkel baseline:

- Median baseline
- Ridge Regression
- Random Forest
- Gradient Boosting

Baseline

Baseline-modellen predikerer alltid medianprisen fra treningsdataene.

Resultatet ble:

- RMSE: 75 702,64
- MAE: 26 547,90
- R^2: -0,0305

Baseline-modellen er veldig enkel, men den er nyttig som et sammenligningsgrunnlag. Hvis en mer avansert modell ikke klarer å gjøre det bedre enn baseline, ville det vært et tegn på at modellen ikke lærer nok nyttig informasjon.

Ridge Regression

Den første maskinlæringsmodellen jeg testet var Ridge Regression.

Resultatene ble:

- RMSE: 68 763,88
- MAE: 21 142,92
- R^2: 0,1497
- Treningstid: omtrent 2,13 sekunder

Ridge gjorde det klart bedre enn baseline, men siden modellen er lineær forventet jeg at den kunne ha begrensninger på mer kompliserte sammenhenger.

Random Forest

Den neste modellen jeg testet var Random Forest.

Resultatene ble:

- RMSE: 70 600,90
- MAE: 20 980,76
- R^2: 0,1037
- Treningstid: omtrent 5,55 sekunder

Random Forest gjorde det bedre enn baseline, men den hadde høyere RMSE enn både Ridge Regression og Gradient Boosting.

Random Forest var også den modellen som tok lengst tid å trene.

Gradient Boosting

Gradient Boosting var modellen som fikk de beste resultatene.

Resultatene ble:

- RMSE: 68 053,20
- MAE: 19 575,11
- R^2: 0,1672
- Treningstid: omtrent 1,45 sekunder

Denne modellen hadde lavest RMSE og MAE og samtidig høyest R^2. Jeg valgte derfor Gradient Boosting som den endelige modellen som brukes i nettsiden.

Sammenligning av modellene

Modell	RMSE	MAE	R^2
Gradient Boosting	68 053,20	19 575,11	0,1672
Ridge Regression	68 763,88	21 142,92	0,1497
Random Forest	70 600,90	20 980,76	0,1037
Median baseline	75 702,64	26 547,90	-0,0305

Gradient Boosting fikk best resultat av modellene jeg testet, Selv om dette var bedre enn baseline og de andre modellene, er spesielt R^2-resultatet ganske lavt. Modellen forklarer derfor bare en forholdsvis liten del av forskjellene mellom bilprisene.

Jeg tror dette delvis skyldes at datasettet ikke inneholder all informasjon som vanligvis påvirker bruktbilpris. Jeg hadde for eksempel ikke detaljert informasjon om bilens faktiske tilstand, servicehistorikk, ekstrautstyr eller geografisk plassering.

En annen ting jeg la merke til var forskjellen mellom RMSE og den typiske prediksjonsfeilen. Median absolutt feil var omtrent 8 858 dollar, mens RMSE var hele 68 053 dollar. Da jeg undersøkte feilene nærmere fant jeg at enkelte svært dyre biler ga ekstremt store feil.

Spesielt la jeg merke til at 50 biler hadde nøyaktig samme maksimalpris på 2 954 083 dollar. Modellen predikerte mye lavere priser for flere av disse bilene. Siden RMSE straffer store feil mye mer enn små feil, ser disse bilene ut til å være en viktig grunn til at RMSE ble så høy.

Derfor synes jeg RMSE bør ses sammen med MAE og fordelingen av prediksjonsfeilene.

Feature importance

Jeg brukte permutation importance for å undersøke hvilke egenskaper som hadde størst betydning for modellen.

Resultatene ble:

Egenskap	Importance
Kilometerstand	5579,42
Motorstørrelse	1777,22
Hestekrefter	1499,34
Årsmodell	1324,72
Merke	687,60
Girkasse	432,59
Modell	233,74
Sylindre	65,62
Skadehistorikk	42,68
Clean title	0
Drivstofftype	-7,35

Da jeg testet permutation importance, var kilometerstand klart den viktigste variabelen. Dette var ikke veldig overraskende, siden kilometerstand ofte er en av de første tingene man ser på når man vurderer en bruktbil.

De neste variablene var motorstørrelse, hestekrefter, årsmodell og merke.

Det som overrasket litt mer var at modellnavnet fikk lavere importance enn blant annet motorstørrelse og hestekrefter. Jeg hadde forventet at selve bilmodellen skulle ha større betydning.

Drivstofftype fikk en importance-verdi litt under null. Jeg tolker ikke dette som at drivstofftype har negativ effekt på bilens pris. Det betyr heller at modellen i denne testen ikke mistet noe særlig ytelse da denne variabelen ble stokket tilfeldig.

Dette viser også at feature importance må tolkes forsiktig. Verdiene sier noe om hvordan akkurat denne modellen brukte variablene, ikke nødvendigvis hvor viktig en egenskap er for bruktbilpriser generelt.

Feilanalyse

Jeg undersøkte også hvor modellen gjorde de største feilene.

Medianen for absolutt prediksjonsfeil var omtrent 8 858 dollar.

Omtrent:

- 90 % av prediksjonene hadde en feil under 37 915 dollar
- 95 % hadde en feil under 58 405 dollar

Modellen fungerte mye bedre på biler i normale prisklasser enn på ekstremt dyre biler.

Faktisk pris	Gjennomsnittlig absolutt feil
Under $20 000	$8 336
$20 000–$40 000	$13 296
$40 000–$75 000	$17 070
$75 000–$150 000	$29 171
$150 000–$300 000	$118 375
Over $300 000	$539 910

En interessant ting jeg fant i datasettet var at 50 biler har nøyaktig maksimalprisen på 2 954 083 dollar. Disse bilene førte til noen svært store prediksjonsfeil fordi modellen ofte predikerte langt lavere priser.

Dette forklarer også hvorfor RMSE blir såpass høy. Siden RMSE straffer store feil ekstra mye, kan noen få veldig store feil påvirke totalresultatet kraftig.

Hvis jeg skulle arbeidet videre med prosjektet, ville jeg derfor undersøkt disse ekstremverdiene nærmere. Jeg ville først forsøkt å finne ut om de faktisk representerer realistiske bilpriser, eller om de kan være et resultat av hvordan datasettet er laget.

4: DEPLOYMENT

Den endelige modellen ble gjort tilgjengelig gjennom en nettside laget med Streamlit.

På nettsiden kan brukeren fylle inn:

- Merke
- Modell
- Årsmodell
- Kilometerstand
- Drivstofftype
- Girkasse
- Hestekrefter
- Motorstørrelse
- Antall sylindre
- Skadehistorikk
- Clean title

Når brukeren trykker på Estimate price, blir verdiene samlet i en Pandas DataFrame. Denne sendes gjennom den samme preprocessing-pipelinen som ble brukt under treningen. Deretter bruker Gradient Boosting-modellen dataene til å gi et prisestimat.

Et eksempel jeg testet i nettsiden var:

- Toyota Yaris Base
- Årsmodell 2008
- Kilometerstand 153 000
- Diesel
- 90 hestekrefter
- 1,5 liters motor
- 3 sylindre
- Ingen rapportert skade
- Clean title

Nettsiden estimerte da prisen til omtrent 13 333 dollar.

Nettsiden viser også hvilken modell som brukes og resultatene modellen fikk på testsettet:

- Modell: Gradient Boosting
- RMSE: 68 053,20
- MAE: 19 575,11
- R^2: 0,1672

Den ferdigtrente modellen ble lagret med Joblib. Dette gjør at nettsiden kan laste modellen direkte når den starter, i stedet for å trene hele modellen på nytt hver gang.

Nettsiden viser også tydelig at resultatet bare er et maskinlæringsestimat og ikke en garantert markedspris.

Monitorering og videre forbedringer

Hvis jeg skulle fortsatt prosjektet, ville det første jeg undersøkte vært de ekstreme prisverdiene. De 50 bilene med en pris på nøyaktig 2 954 083 dollar skiller seg veldig ut fra resten av datasettet og ser ut til å påvirke RMSE mye.

Jeg ville først undersøkt om disse verdiene faktisk er ment å være realistiske priser eller om de kommer av måten Kaggle-datasettet er laget på. Hvis de viser seg å være spesielle eller kunstige verdier, kunne det vært aktuelt å teste modellen både med og uten disse bilene og sammenligne resultatene.

Det neste jeg ville gjort er å forbedre inputvariablene. Informasjon om bilens tilstand, servicehistorikk, ekstrautstyr og geografisk område kunne sannsynligvis gjort det lettere å forklare forskjellene i pris.

Etter å ha undersøkt ekstremverdiene ville jeg testet om hyperparameter-tuning på Gradient Boosting kunne forbedret resultatene ytterligere. I dette prosjektet sammenlignet jeg tre modeller, men jeg brukte ikke mye tid på å optimalisere hver enkelt modell.

En annen forbedring kunne vært å vise et prisintervall i stedet for én bestemt pris. Når nettsiden for eksempel viser 13 333 dollar, kan tallet virke mer presist enn modellen egentlig er. Et resultat som «omtrent 11 000–15 000 dollar» kunne vært mer realistisk for brukeren.

Hvis løsningen faktisk skulle vært brukt over lengre tid, måtte modellen også blitt testet på nyere data med jevne mellomrom. Bruktbilmarkedet endrer seg, og en modell som fungerer bra på dagens data trenger ikke å fungere like bra senere.

Dokumentasjon av applikasjonen

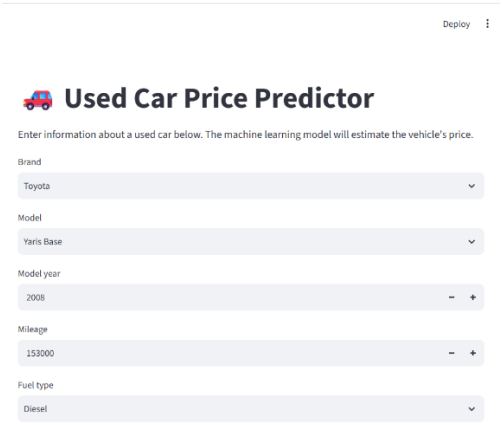
GitHub repository:

https://github.com/M593778/ML-assignment-2--Project-work.git

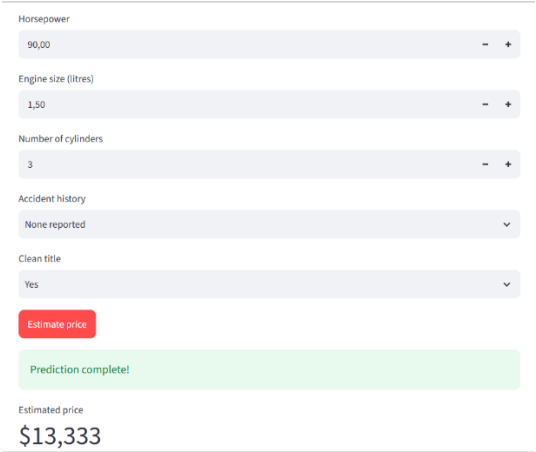
Live nettside:

https://ml-assignment-2--project-work-7zgkf6x9hdxyf99pj2mmtg.streamlit.app/

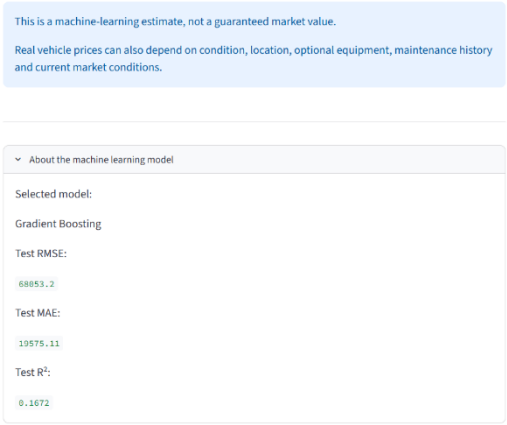
Figur 1: Brukergrensesnittet til nettsiden



Figur 2: Et eksempel på en ferdig prediksjon



Figur 3: Informasjon om Gradient Boosting-modellen og evalueringsresultatene



5: REFERANSER

Kaggle. *Regression of Used Car Prices – Playground Series, Season 4, Episode 9.*

Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3. utgave.

Scikit-learn dokumentasjon. Brukt til preprocessing, maskinlæringsmodeller, pipelines, evalueringsmetrikker og permutation importance.

Streamlit dokumentasjon. Brukt til å utvikle og publisere nettsiden.

Forelesningsmateriale fra DAT158.

OpenAI ChatGPT. Brukt til hjelp med kodeoppsett, feilsøking og forklaringer. Kode, resultater og nettsiden ble testet før innlevering.